

Comparaison de formulations du problème du voyageur de commerce

Ivan Lejeune

20 avril 2026

- Problème du voyageur de commerce (TSP)
- Trouver le plus court cycle hamiltonien
- Problème NP-difficile
- Objectif : comparer différentes formulations MILP

Définition du problème

- Graphe complet avec distances c_{ij}

- Variables :

$$x_{ij} = \begin{cases} 1 & \text{si l'arc est utilisé} \\ 0 & \text{sinon} \end{cases}$$

- Contraintes :

Une entrée par sommet

Une sortie par sommet

- Variables supplémentaires : y_i
- Interprétation : ordre de visite
- Élimination des sous-tours

$$x_{ij}(n-1) + y_i - y_j \leq n-2$$

- Formulation compacte
- Relaxation linéaire faible

- Variables : q_{ij} (flux)
- Idée : envoyer $n - 1$ unités depuis le sommet 1
- Chaque sommet consomme 1 unité
- Assure la connectivité globale
- Relaxation plus forte

$$MTZ = \begin{cases} \min z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \\ \sum_{i,j} x_{ij} = 1 \quad \forall i \in \{1, \dots, n\} \\ \sum_{i,j} x_{ij} = 1 \quad \forall j \in \{1, \dots, n\} \\ x_{ij}(n-1) + y_i - y_j \leq n-2 \quad \forall i, j \in \{1, \dots, n\}, i \neq j \\ x_{ij} \in \{0, 1\} \quad \forall i, j \in \{1, \dots, n\} \\ y_i \geq 0 \quad \forall i \in \{1, \dots, n\} \end{cases}$$

$$SF = \left\{ \begin{array}{l} \min z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \\ \sum_{i,j} x_{ij} = 1 \quad \forall i \in \{1, \dots, n\} \\ \sum_{i,j} x_{ij} = 1 \quad \forall j \in \{1, \dots, n\} \\ q_{ij} \leq (n-1)x_{ij} \quad \forall i, j \in \{1, \dots, n\}, i \neq j \\ \sum_{j=1}^n q_{1j} = n-1 \\ \sum_{i \neq j} q_{ji} - \sum_{i \neq j} q_{ij} = 1 \quad \forall j \in \{2, \dots, n\} \\ x_{ij} \in \{0, 1\} \quad \forall i, j \in \{1, \dots, n\} \\ 0 \leq q_{ij} \leq n-1 \quad \forall i, j \in \{1, \dots, n\} \end{array} \right.$$

MTZ	SF
Simple	Plus complexe
Peu de variables	Plus de variables
Relaxation faible	Relaxation forte

Conclusion : SF devrait être plus performante

- Langage : Julia
- Librairie : JuMP + Cbc
- Parsing des instances à partir des tableaux de distances
- Implémentation modulaire :
 - `solve_mtz`
 - `solve_sf`

Le code MTZ - 1

```
function solve_mtz(c; time_limit=300.0, use_eq2=false)
    n = size(c,1)
    model = Model(Cbc.Optimizer)

    set_optimizer_attribute(model, "seconds", time_limit)

    @variable(model, x[1:n,1:n], Bin)
    @variable(model, y[1:n])

    @objective(model, Min,
        sum(c[i,j]*x[i,j] for i in 1:n, j in 1:n))

    @constraint(model, [i=1:n],
        sum(x[i,j] for j in 1:n if j != i) == 1)
    @constraint(model, [j=1:n],
        sum(x[i,j] for i in 1:n if i != j) == 1)

    @constraint(model, [i=1:n], x[i,i] == 0)
```

Le code MTZ - 2

```
@constraint(model, [i=2:n, j=2:n; i != j],  
    x[i,j]*(n-1) + y[i] - y[j] <= n-2  
)  
  
@constraint(model, y[1] == 0)  
@constraint(model, [i=2:n], 1 <= y[i] <= n-1)  
  
if use_eq2  
    @constraint(model, [i=1:n, j=i+1:n], x[i,j] + x[j,i] <= 1)  
end  
  
optimize!(model)  
  
return (  
    value = has_values(model) ? objective_value(model) : NaN,  
    time = solve_time(model),  
    status = termination_status(model)  
)  
end
```

Le code SF - 1

```
function solve_sf(c; time_limit=300.0, use_eq2=false)
    n = size(c,1)
    model = Model(Cbc.Optimizer)

    set_optimizer_attribute(model, "seconds", time_limit)

    @variable(model, x[1:n,1:n], Bin)
    @variable(model, 0 <= q[1:n,1:n] <= n-1)

    @objective(model, Min,
        sum(c[i,j]*x[i,j] for i in 1:n, j in 1:n))

    @constraint(model, [i=1:n],
        sum(x[i,j] for j in 1:n if j != i) == 1)
    @constraint(model, [j=1:n],
        sum(x[i,j] for i in 1:n if i != j) == 1)

    @constraint(model, [i=1:n], x[i,i] == 0)
    @constraint(model, [i=1:n], q[i,i] == 0)
```

Le code SF - 2

```
@constraint(model, [i=1:n, j=1:n; i != j],
    q[i,j] <= (n-1)*x[i,j]
)

@constraint(model, sum(q[1,j] for j in 2:n) == n-1)

@constraint(model, [j=2:n], sum(q[i,j] for i in 1:n if i != j)
    - sum(q[j,k] for k in 1:n if k != j) == 1
)

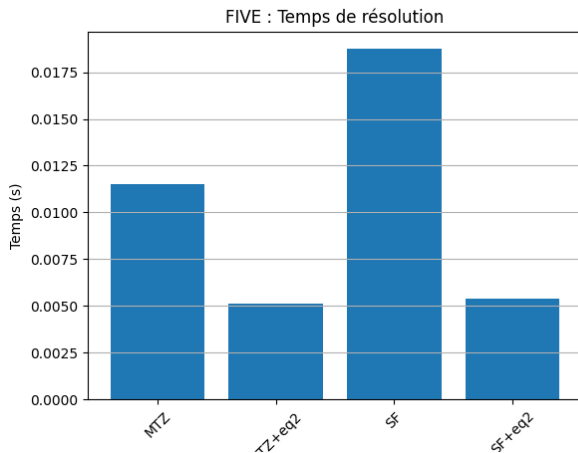
if use_eq2
    @constraint(model, [i=1:n, j=i+1:n], x[i,j] + x[j,i] <= 1)
end
optimize!(model)
return (
    value = has_values(model) ? objective_value(model) : NaN,
    time = solve_time(model), status = termination_status(model)
)
end
```

- Calcul des relaxations pour plusieurs instances
- Observation :
 - SF donne des bornes plus élevées
 - Relaxation plus serrée

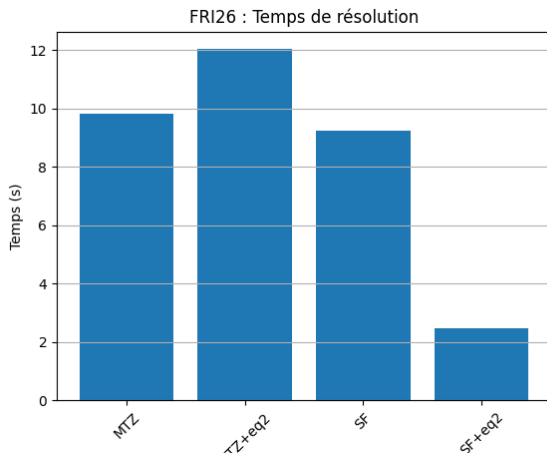
Impact : meilleure efficacité du branch-and-bound

Résultats expérimentaux — 1

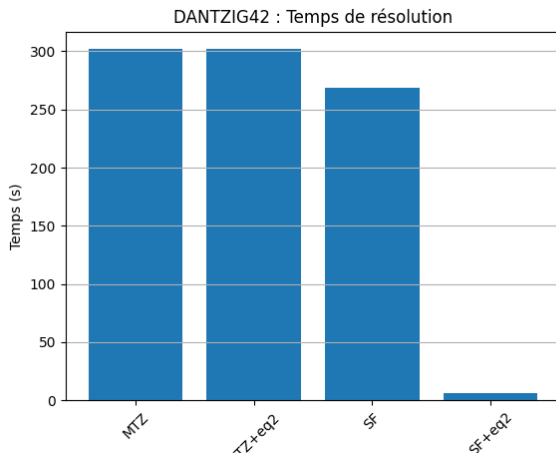
- Comparaison des temps de calcul
- Comparaison des solutions obtenues



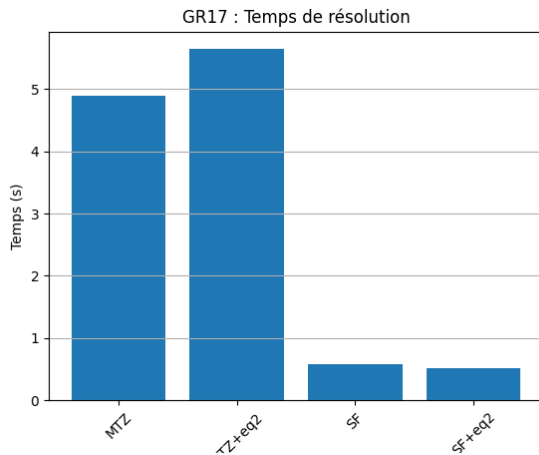
Résultats expérimentaux — 2



Résultats expérimentaux — 3



Résultats expérimentaux — 4



- SF plus rapide que MTZ sauf sur les plus petites instances
- Écart de temps plus important sur certaines instances
- Importance de la relaxation linéaire

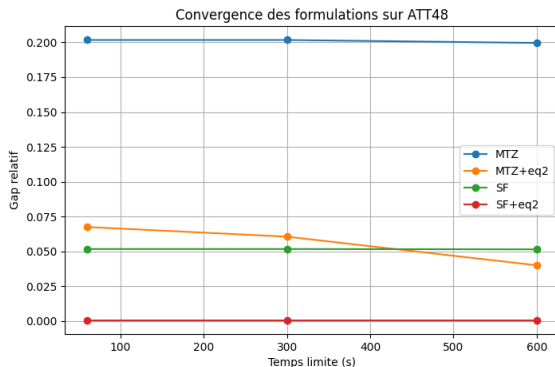
Ajout de l'inégalité $x_{ij} + x_{ji} \leq 1$

- Élimine les arcs bidirectionnels
- Réduit les solutions fractionnaires
- Améliore la relaxation

Résultat : amélioration des performances

Test sur ATT48

- Instance plus grande, solution à 33523.
- Étude de la convergence en temps



- Modélisation MTZ (le problème de l'infaisabilité)
- Parsing des données
- Temps de calcul
- Compréhension des relaxations

- SF plus performante que MTZ
- La relaxation linéaire est un facteur clé
- Le rajout de contraintes supplémentaires améliore les performances

Ouverture :

- Autres formulations (DFJ)
- Rajout de contraintes généralisées

Questions ?